

```

/* Includes -----*/
#include "main.h"
#include <stdio.h>
#include "R307.h"
#include "ILI9341_STM32_Driver.h"
#include "ILI9341_GFX.h"
#include <string.h>

/* USER CODE BEGIN Includes */
#define EEPROM_I2C_ADDR (0x50 << 1)
#define ADDR_VOTE1 0x0000
#define ADDR_VOTE2 0x0004
#define ADDR_VOTE3 0x0008
#define ADDR_VOTED_FLAGS_BASE 0x0100
/* USER CODE END Includes */

/* USER CODE BEGIN PD */
uint32_t vote1 = 0;
uint32_t vote2 = 0;
uint32_t vote3 = 0;
uint8_t has_voted[256] = {0};
/* USER CODE END PD */

/* USER CODE BEGIN PM */
typedef enum
{
    IDLE_STATE = 0,
    VOTED_STATE
} VotingState;
VotingState voteState = IDLE_STATE;
/* USER CODE END PM */

/* Private variables -----*/
I2C_HandleTypeDef hi2c1;
RTC_HandleTypeDef hrtc;
SPI_HandleTypeDef hspi1;
DMA_HandleTypeDef hdma_spi1_tx;
UART_HandleTypeDef huart1;
UART_HandleTypeDef huart2;

/* Private function prototypes -----*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_DMA_Init(void);
static void MX_I2C1_Init(void);
static void MX_SPI1_Init(void);
static void MX_USART1_UART_Init(void);
static void MX_USART2_UART_Init(void);
static void MX_RTC_Init(void);

/* USER CODE BEGIN PFP */
#define NUM_FINGERS 2
uint8_t Check_If_Voted(uint16_t id);
void Elect_start(void);
void beep(void);
void showResult(void);
void EEPROM_ResetVotes(void);
void enroll_Fingerprints(void);
void Clearing_Fingerprints(void);
void Mark_As_Voted(uint16_t id);
void EEPROM_Write(uint16_t memAddr, uint32_t data);
uint32_t EEPROM_Read(uint16_t memAddr);
/* USER CODE END PFP */

int _io_putchar(int ch)
{
    HAL_UART_Transmit(&huart2, (uint8_t *)&ch, 1, HAL_MAX_DELAY);
    return ch;
}

int main(void)
{
    HAL_Init();
    SystemClock_Config();

    /* Initialize all configured peripherals */
    MX_GPIO_Init();

```

```

MX_DMA_Init();
MX_I2C1_Init();
MX_SPI1_Init();
MX_USART1_UART_Init();
MX_USART2_UART_Init();
MX_RTC_Init();

/* USER CODE BEGIN 2 */
ILI9341_Init();
ILI9341_SetRotation(SCREEN_VERTICAL_2);

vote1 = EEPROM_Read(ADDR_VOTE1);
vote2 = EEPROM_Read(ADDR_VOTE2);
vote3 = EEPROM_Read(ADDR_VOTE3);

ILI9341_FillScreen(BLACK);
ILI9341_DrawText("BIOMETRIC VOTING", FONT6, 25, 100, WHITE, BLACK);
ILI9341_DrawText("MACHINE", FONT6, 100, 140, WHITE, BLACK);

printf("\r\n===== VOTING MACHINE =====\r\n");
printf("Restored Votes: \r\n");
printf("C1=%lu C2=%lu C3=%lu\r\n", vote1, vote2, vote3);
HAL_Delay(10000);
beep();

if (R307_VerifyPassword(0x00000000) != HAL_OK)
{
    ILI9341_FillScreen(WHITE);
    ILI9341_DrawText("BIOMETRIC VOTING", FONT6, 25, 10, BLACK, WHITE);
    ILI9341_DrawText("MACHINE", FONT6, 100, 30, BLACK, WHITE);
    ILI9341_DrawText("password verification failed", FONT7, 15, 120, RED, WHITE);
    printf("Password verify failed!");
    HAL_Delay(4000);

    ILI9341_FillScreen(BLACK);
    ILI9341_DrawText("WARNING!!!", FONT6, 90, 100, RED, BLACK);
    ILI9341_DrawText("Error_r307", FONT7, 120, 140, RED, BLACK);
    beep();
    Error_Handler();
}

if (HAL_RTCEX_BKUPRead(&hrtc, RTC_BKP_DR0) != 0x9876)
{
    HAL_RTCEX_BKUPWrite(&hrtc, RTC_BKP_DR0, 0x9876);
}

uint8_t start_button_pressed = 0;

while (1)
{
    if (HAL_GPIO_ReadPin(START_GPIO_Port, START_Pin) == GPIO_PIN_RESET)
    {
        if (start_button_pressed == 0)
        {
            ILI9341_FillScreen(WHITE);
            ILI9341_DrawText("BIOMETRIC VOTING", FONT6, 25, 10, BLACK, WHITE);
            ILI9341_DrawText("MACHINE", FONT6, 100, 30, BLACK, WHITE);
            ILI9341_DrawText("START VOTING ", FONT6, 60, 120, BLUE, WHITE);
            printf("\r\n---START BUTTON PRESSED---\r\n");
            HAL_Delay(2000);
            Elect_start();
            start_button_pressed = 1;
            HAL_Delay(200); // Debounce delay
        }
    }
    else
    {
        start_button_pressed = 0;
    }

    if (HAL_GPIO_ReadPin(RESULT_GPIO_Port, RESULT_Pin) == GPIO_PIN_RESET)
    {
        showResult();
        HAL_Delay(1000);
        while (HAL_GPIO_ReadPin(RESULT_GPIO_Port, RESULT_Pin) == GPIO_PIN_RESET);
    }
    if (HAL_GPIO_ReadPin(RSTEEPROM_GPIO_Port, RSTEEPROM_Pin) == GPIO_PIN_RESET)

```

```

    {
        EEPROM_ResetVotes();
        HAL_Delay(300); // 3 sec long press indicator
        vote1 = vote2 = vote3 = 0;
        printf("EEPROM RESET DONE\r\n");
        ILI9341_FillScreen(WHITE);
        ILI9341_DrawText("BIOMETRIC VOTING", FONT6, 25, 10, BLACK, WHITE);
        ILI9341_DrawText("MACHINE", FONT6, 100, 30, BLACK, WHITE);
        ILI9341_DrawText("EEPROM RESET DONE", FONT7, 40, 120, BLUE, WHITE);
        while (HAL_GPIO_ReadPin(RSTEEPROM_GPIO_Port, RSTEEPROM_Pin) == GPIO_PIN_RESET);
    }

    if (HAL_GPIO_ReadPin(FIGER_ENROLL_GPIO_Port, FIGER_ENROLL_Pin) == GPIO_PIN_RESET)
    {
        enroll_Fingerprints();
        HAL_Delay(1000);
        while (HAL_GPIO_ReadPin(FIGER_ENROLL_GPIO_Port, FIGER_ENROLL_Pin) == GPIO_PIN_RESET);
    }

    if (HAL_GPIO_ReadPin(FINGER_REMOVE_GPIO_Port, FINGER_REMOVE_Pin) == GPIO_PIN_RESET)
    {
        Clearing_Fingerprints();
        HAL_Delay(1000);
        while (HAL_GPIO_ReadPin(FINGER_REMOVE_GPIO_Port, FINGER_REMOVE_Pin) == GPIO_PIN_RESET);
    }
}

void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

    __HAL_RCC_PWR_CLK_ENABLE();
    HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE2);

    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_LSI|RCC_OSCILLATORTYPE_HSE;
    RCC_OscInitStruct.HSEState = RCC_HSE_ON;
    RCC_OscInitStruct.LSIState = RCC_LSI_ON;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
    RCC_OscInitStruct.PLL.PLLM = 25;
    RCC_OscInitStruct.PLL.PLLN = 168;
    RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
    RCC_OscInitStruct.PLL.PLLQ = 4;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
    {
        Error_Handler();
    }

    RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYCLK
                                |RCC_CLOCKTYPE_PCLK1 | RCC_CLOCKTYPE_PCLK2;
    RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
    RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
    if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
    {
        Error_Handler();
    }
}

static void MX_I2C1_Init(void)
{
    hi2c1.Instance = I2C1;
    hi2c1.Init.ClockSpeed = 400000;
    hi2c1.Init.DutyCycle = I2C_DUTYCYCLE_2;
    hi2c1.Init.OwnAddress1 = 0;
    hi2c1.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
    hi2c1.Init.DualAddressMode = I2C_DUALADDRESS_DISABLE;
    hi2c1.Init.OwnAddress2 = 0;
    hi2c1.Init.GeneralCallMode = I2C_GENERALCALL_DISABLE;
    hi2c1.Init.NoStretchMode = I2C_NOSTRETCH_DISABLE;
    if (HAL_I2C_Init(&hi2c1) != HAL_OK)
    {
        Error_Handler();
    }
}

```

```

}

static void MX_RTC_Init(void)
{
    hrtc.Instance = RTC;
    hrtc.Init.HourFormat = RTC_HOURFORMAT_24;
    hrtc.Init.AsynchPrediv = 127;
    hrtc.Init.SynchPrediv = 255;
    hrtc.Init.OutPut = RTC_OUTPUT_DISABLE;
    hrtc.Init.OutPutPolarity = RTC_OUTPUT_POLARITY_HIGH;
    hrtc.Init.OutPutType = RTC_OUTPUT_TYPE_OPENDRAIN;
    if (HAL_RTC_Init(&hrtc) != HAL_OK)
    {
        Error_Handler();
    }
}

static void MX_SPI1_Init(void)
{
    hspi1.Instance = SPI1;
    hspi1.Init.Mode = SPI_MODE_MASTER;
    hspi1.Init.Direction = SPI_DIRECTION_2LINES;
    hspi1.Init.DataSize = SPI_DATASIZE_8BIT;
    hspi1.Init.CLKPolarity = SPI_POLARITY_LOW;
    hspi1.Init.CLKPhase = SPI_PHASE_1EDGE;
    hspi1.Init.NSS = SPI_NSS_SOFT;
    hspi1.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_4;
    hspi1.Init.FirstBit = SPI_FIRSTBIT_MSB;
    hspi1.Init.TIMode = SPI_TIMODE_DISABLE;
    hspi1.Init.CRCCalculation = SPI_CRCCALCULATION_DISABLE;
    hspi1.Init.CRCPolynomial = 10;
    if (HAL_SPI_Init(&hspi1) != HAL_OK)
    {
        Error_Handler();
    }
}

static void MX_USART1_UART_Init(void)
{
    huart1.Instance = USART1;
    huart1.Init.BaudRate = 57600;
    huart1.Init.WordLength = UART_WORDLENGTH_8B;
    huart1.Init.StopBits = UART_STOPBITS_1;
    huart1.Init.Parity = UART_PARITY_NONE;
    huart1.Init.Mode = UART_MODE_TX_RX;
    huart1.Init.HwFlowCtl = UART_HWCONTROL_NONE;
    huart1.Init.OverSampling = UART_OVERSAMPLING_16;
    if (HAL_UART_Init(&huart1) != HAL_OK)
    {
        Error_Handler();
    }
}

static void MX_USART2_UART_Init(void)
{
    huart2.Instance = USART2;
    huart2.Init.BaudRate = 115200;
    huart2.Init.WordLength = UART_WORDLENGTH_8B;
    huart2.Init.StopBits = UART_STOPBITS_1;
    huart2.Init.Parity = UART_PARITY_NONE;
    huart2.Init.Mode = UART_MODE_TX_RX;
    huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
    huart2.Init.OverSampling = UART_OVERSAMPLING_16;
    if (HAL_UART_Init(&huart2) != HAL_OK)
    {
        Error_Handler();
    }
}

static void MX_DMA_Init(void)
{
    __HAL_RCC_DMA2_CLK_ENABLE();
    HAL_NVIC_SetPriority(DMA2_Stream3_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(DMA2_Stream3_IRQn);
}

static void MX_GPIO_Init(void)

```

```

{
    GPIO_InitTypeDef GPIO_InitStruct = {0};

    __HAL_RCC_GPIOC_CLK_ENABLE();
    __HAL_RCC_GPIOH_CLK_ENABLE();
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_GPIOB_CLK_ENABLE();

    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0|GPIO_PIN_1|EXTRA_OUT_Pin|GPIO_PIN_10
        |C1LED_Pin|C2LED_Pin|C3LED_Pin|EEROMLED_Pin, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(BUZZER_GPIO_Port, BUZZER_Pin, GPIO_PIN_RESET);

    GPIO_InitStruct.Pin = C1_Pin|C2_Pin|C3_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
    GPIO_InitStruct.Pull = GPIO_PULLUP;
    HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);

    GPIO_InitStruct.Pin = RESULT_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
    GPIO_InitStruct.Pull = GPIO_PULLUP;
    HAL_GPIO_Init(RESULT_GPIO_Port, &GPIO_InitStruct);

    GPIO_InitStruct.Pin = GPIO_PIN_0|GPIO_PIN_1|EXTRA_OUT_Pin|GPIO_PIN_10
        |C1LED_Pin|C2LED_Pin|C3LED_Pin|EEROMLED_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

    GPIO_InitStruct.Pin = BUZZER_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(BUZZER_GPIO_Port, &GPIO_InitStruct);

    GPIO_InitStruct.Pin = FINGER_REMOVE_Pin|FINGER_ENROLL_Pin|EXTRA_Pin|START_Pin|RSTEEPROM_Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
    GPIO_InitStruct.Pull = GPIO_PULLUP;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
}

/* USER CODE BEGIN 4 */
void Elect_start(void)
{
    uint16_t matched_id, score;
    HAL_StatusTypeDef status;
    uint8_t retryCount = 0;
    const uint8_t MAX_RETRY = 3;
    uint8_t verified = 0;
    uint8_t finger = 0;
    uint8_t rxData = 0;

    ILI9341_FillScreen(WHITE);
    ILI9341_DrawText("BIOMETRIC VOTING ", FONT6, 25, 10, BLACK, WHITE);
    ILI9341_DrawText("MACHINE", FONT6, 100, 30, BLACK, WHITE);
    ILI9341_DrawText("Place finger to verify", FONT7, 20, 120, BLUE, WHITE);

    char buffer[50];
    sprintf(buffer, "(Max %d attempts)", MAX_RETRY);
    ILI9341_DrawText(buffer, FONT7, 60, 150, BLUE, WHITE);
    printf("\r\nPlace finger to verify... (Max %d attempts)\r\n", MAX_RETRY);

    while (retryCount < MAX_RETRY && !verified)
    {
        status = R307_FingerVerify(&matched_id, &score); // Standard interface inference
        if (status == HAL_OK)
        {
            if (Check_If_Voted(matched_id) == 1)
            {
                printf("\r\n>>> Finger ID %d has ALREADY VOTED!\r\n", matched_id);
                ILI9341_FillScreen(WHITE);
                ILI9341_DrawText("BIOMETRIC VOTING", FONT6, 25, 10, BLACK, WHITE);
                ILI9341_DrawText("MACHINE", FONT6, 100, 30, BLACK, WHITE);
                ILI9341_DrawText("ALREADY VOTED!", FONT7, 30, 120, RED, WHITE);
                HAL_Delay(3000);
                return;
            }
        }
    }
}

```

```

        printf("\r\n>>> Finger Verified Successfully! ID: %d", matched_id);
        ILI9341_FillScreen(WHITE);
        ILI9341_DrawText("BIOMETRIC VOTING", FONT6, 25, 10, BLACK, WHITE);
        ILI9341_DrawText("MACHINE", FONT6, 100, 30, BLACK, WHITE);
        ILI9341_DrawText("Finger Successfull", FONT7, 30, 120, BLUE, WHITE);
        HAL_Delay(3000);
        verified = 1;
        finger = 1;
        break;
    }

    retryCount++;
    ILI9341_FillScreen(WHITE);
    ILI9341_DrawText("BIOMETRIC VOTING", FONT6, 25, 10, BLACK, WHITE);
    ILI9341_DrawText("MACHINE", FONT6, 100, 30, BLACK, WHITE);
    ILI9341_DrawText("Verification Failed! ", FONT7, 20, 120, RED, WHITE);
    char buffer1[50];
    sprintf(buffer1, "Attempt %d/%d", retryCount, MAX_RETRY);
    ILI9341_DrawText(buffer1, FONT7, 60, 150, RED, WHITE);
    printf("\r\nVerification Failed! Attempt %d/%d", retryCount, MAX_RETRY);
    HAL_Delay(5000);
}

if (!verified)
{
    ILI9341_FillScreen(WHITE);
    ILI9341_DrawText("BIOMETRIC VOTING", FONT6, 25, 10, BLACK, WHITE);
    ILI9341_DrawText("MACHINE", FONT6, 100, 30, BLACK, WHITE);
    ILI9341_DrawText("Verification Failed", FONT7, 30, 100, RED, WHITE);
    printf("\r\nVerification failed. Waiting for manual override...\r\n");
    HAL_Delay(3000);

    /* WAIT HERE FOR EXTRA BUTTON */
    while (1)
    {
        if (HAL_GPIO_ReadPin(EXTRA_GPIO_Port, EXTRA_Pin) == GPIO_PIN_RESET)
        {
            ILI9341_FillScreen(WHITE);
            ILI9341_DrawText("BIOMETRIC VOTING", FONT6, 25, 10, BLACK, WHITE);
            ILI9341_DrawText("MACHINE", FONT6, 100, 30, BLACK, WHITE);
            ILI9341_DrawText("Manual override accepted", FONT7, 20, 100, BLUE, WHITE);
            printf("\r\nManual override accepted\r\n");
            HAL_Delay(3000);
            break;
        }
    }
}

if (finger == 1)
{
    uint32_t faceStart = HAL_GetTick();
    uint8_t faceVerified = 0;
    int lastRemainingSeconds = -1;

    ILI9341_FillScreen(WHITE);
    ILI9341_DrawText("BIOMETRIC VOTING", FONT6, 25, 10, BLACK, WHITE);
    ILI9341_DrawText("MACHINE", FONT6, 100, 30, BLACK, WHITE);
    ILI9341_DrawText("Waiting for Face", FONT7, 60, 110, BLUE, WHITE);
    ILI9341_DrawText("Verification...", FONT7, 60, 140, BLUE, WHITE);
    printf("Waiting for Face Verification...\r\n");

    while (HAL_GetTick() - faceStart < 60000)
    {
        uint32_t elapsed = HAL_GetTick() - faceStart;
        int remainingSeconds = 60 - (elapsed / 1000);

        if (remainingSeconds != lastRemainingSeconds)
        {
            char timerBuf[20];
            sprintf(timerBuf, "Time Left: %02ds", remainingSeconds);
            ILI9341_DrawText(timerBuf, FONT7, 75, 180, RED, WHITE);
            lastRemainingSeconds = remainingSeconds;
        }

        if (HAL_UART_Receive(&huart2, &rxData, 1, 100) == HAL_OK)
        {
            if (rxData == 'Y')

```

```

        {
            faceVerified = 1;
            break;
        }
    }
}

if (faceVerified)
{
    ILI9341_FillScreen(WHITE);
    ILI9341_DrawText("BIOMETRIC VOTING ", FONT6, 25, 10, BLACK, WHITE);
    ILI9341_DrawText("MACHINE", FONT6, 100, 30, BLACK, WHITE);
    ILI9341_DrawText("Face verified", FONT7, 50, 120, DARKGREEN, WHITE);
    printf("Face ID Verified\r\n");
    beep();
    HAL_Delay(3000);
}
else
{
    ILI9341_FillScreen(WHITE);
    ILI9341_DrawText("BIOMETRIC VOTING ", FONT6, 25, 10, BLACK, WHITE);
    ILI9341_DrawText("MACHINE", FONT6, 100, 30, BLACK, WHITE);
    ILI9341_DrawText("Face verification", FONT7, 40, 110, RED, WHITE);
    ILI9341_DrawText("FAILED", FONT7, 100, 140, RED, WHITE);
    printf("Face ID Failed\r\n");
    HAL_Delay(3000);

    while (1)
    {
        if (HAL_GPIO_ReadPin(EXTRA_GPIO_Port, EXTRA_Pin) == GPIO_PIN_RESET)
        {
            ILI9341_FillScreen(WHITE);
            ILI9341_DrawText("BIOMETRIC VOTING ", FONT6, 25, 10, BLACK, WHITE);
            ILI9341_DrawText("MACHINE", FONT6, 100, 30, BLACK, WHITE);
            ILI9341_DrawText("Manual override accepted", FONT7, 20, 100, BLUE, WHITE);
            printf("\r\nManual override accepted\r\n");
            HAL_Delay(3000);
            break;
        }
    }
}

ILI9341_FillScreen(WHITE);
ILI9341_DrawText("BIOMETRIC VOTING", FONT6, 25, 10, BLACK, WHITE);
ILI9341_DrawText("MACHINE", FONT6, 100, 30, BLACK, WHITE);
ILI9341_DrawText("Verification Successfull", FONT7, 30, 120, BLUE, WHITE);
beep();
HAL_Delay(3000);

uint32_t startTime = HAL_GetTick();
uint8_t voted = 0;

ILI9341_FillScreen(WHITE);
ILI9341_DrawText("BIOMETRIC VOTING", FONT6, 25, 10, BLACK, WHITE);
ILI9341_DrawText("MACHINE", FONT6, 100, 30, BLACK, WHITE);
ILI9341_DrawText("Waiting for your vote... ", FONT7, 10, 120, BLUE, WHITE);
printf("\r\nwaiting for your vote... (10 Seconds)");

while ((HAL_GetTick() - startTime < 10000) && !voted)
{
    if (HAL_GPIO_ReadPin(C1_GPIO_Port, C1_Pin) == GPIO_PIN_RESET)
    {
        vote1++;
        EEPROM_Write(ADDR_VOTE1, vote1);
        ILI9341_FillScreen(WHITE);
        ILI9341_DrawText("BIOMETRIC VOTING ", FONT6, 25, 10, BLACK, WHITE);
        ILI9341_DrawText("MACHINE", FONT6, 100, 30, BLACK, WHITE);
        ILI9341_DrawText("[VOTED] Candidate 1 ", FONT7, 40, 120, BLUE, WHITE);
        printf("\r\n[VOTED] Candidate 1\r\n");
        HAL_GPIO_WritePin(C1LED_GPIO_Port, C1LED_Pin, GPIO_PIN_SET);
        beep();
        HAL_Delay(1000);
        HAL_GPIO_WritePin(C1LED_GPIO_Port, C1LED_Pin, GPIO_PIN_RESET);
        voted = 1;
    }
    else if (HAL_GPIO_ReadPin(C2_GPIO_Port, C2_Pin) == GPIO_PIN_RESET)

```

```

    {
        vote2++;
        EEPROM_Write(ADDR_VOTE2, vote2);
        ILI9341_FillScreen(WHITE);
        ILI9341_DrawText("BIOMETRIC VOTING ", FONT6, 25, 10, BLACK, WHITE);
        ILI9341_DrawText("MACHINE", FONT6, 100, 30, BLACK, WHITE);
        ILI9341_DrawText("[VOTED] Candidate 2 ", FONT7, 40, 120, BLUE, WHITE);
        printf("\r\n[VOTED] Candidate 2\r\n");
        HAL_GPIO_WritePin(C2LED_GPIO_Port, C2LED_Pin, GPIO_PIN_SET);
        beep();
        HAL_Delay(1000);
        HAL_GPIO_WritePin(C2LED_GPIO_Port, C2LED_Pin, GPIO_PIN_RESET);
        voted = 1;
    }
    else if (HAL_GPIO_ReadPin(C3_GPIO_Port, C3_Pin) == GPIO_PIN_RESET)
    {
        vote3++;
        EEPROM_Write(ADDR_VOTE3, vote3);
        ILI9341_FillScreen(WHITE);
        ILI9341_DrawText("BIOMETRIC VOTING", FONT6, 25, 10, BLACK, WHITE);
        ILI9341_DrawText("MACHINE", FONT6, 100, 30, BLACK, WHITE);
        ILI9341_DrawText("[VOTED] Candidate 3", FONT7, 40, 120, BLUE, WHITE);
        printf("\r\n[VOTED] Candidate 3\r\n");
        HAL_GPIO_WritePin(C3LED_GPIO_Port, C3LED_Pin, GPIO_PIN_SET);
        beep();
        HAL_Delay(1000);
        HAL_GPIO_WritePin(C3LED_GPIO_Port, C3LED_Pin, GPIO_PIN_RESET);
        voted = 1;
    }
}

if (voted)
{
    Mark_As_Voted(matched_id);
    beep();
    ILI9341_FillScreen(WHITE);
    ILI9341_DrawText("BIOMETRIC VOTING", FONT6, 25, 10, BLACK, WHITE);
    ILI9341_DrawText("MACHINE", FONT6, 100, 30, BLACK, WHITE);
    ILI9341_DrawText("Vote Recorded", FONT7, 85, 90, BLUE, WHITE);
    ILI9341_DrawText("THANK YOU", FONT6, 80, 130, DARKGREEN, WHITE);
    printf("Vote recorded. Thank you!\r\n");
}
else
{
    ILI9341_FillScreen(WHITE);
    ILI9341_DrawText("BIOMETRIC VOTING", FONT6, 25, 10, BLACK, WHITE);
    ILI9341_DrawText("MACHINE", FONT6, 100, 30, BLACK, WHITE);
    ILI9341_DrawText("Voting timeout! Try again", FONT7, 10, 120, RED, WHITE);
    printf("\r\nVoting timeout! Try again.\r\n");
}
}

void EEPROM_Write(uint16_t memAddr, uint32_t data)
{
    uint8_t buf[6];
    buf[0] = memAddr >> 8;
    buf[1] = memAddr & 0xFF;
    buf[2] = (data >> 24) & 0xFF;
    buf[3] = (data >> 16) & 0xFF;
    buf[4] = (data >> 8) & 0xFF;
    buf[5] = data & 0xFF;

    HAL_I2C_Master_Transmit(&hi2c1, EEPROM_I2C_ADDR, buf, 6, HAL_MAX_DELAY);
    HAL_Delay(10); // EEPROM write delay
}

uint32_t EEPROM_Read(uint16_t memAddr)
{
    uint8_t addr[2];
    uint8_t data[4];

    addr[0] = memAddr >> 8;
    addr[1] = memAddr & 0xFF;

    HAL_I2C_Master_Transmit(&hi2c1, EEPROM_I2C_ADDR, addr, 2, HAL_MAX_DELAY);
    HAL_I2C_Master_Receive(&hi2c1, EEPROM_I2C_ADDR, data, 4, HAL_MAX_DELAY);
    return ((uint32_t)data[0] << 24) |

```



```

        ((uint32_t)data[1] << 16) |
        ((uint32_t)data[2] << 8) |
        (data[3]);
    }

uint8_t Check_If_Voted(uint16_t id)
{
    uint32_t status = EEPROM_Read(ADDR_VOTED_FLAGS_BASE + (id * 4));
    if (status == 1) {
        return 1;
    }
    return 0;
}

void Mark_As_Voted(uint16_t id)
{
    EEPROM_Write(ADDR_VOTED_FLAGS_BASE + (id * 4), 1);
}

void showResult(void)
{
    printf("\r\n==== RESULT ====\r\n");
    printf("Candidate 1: %lu\r\n", vote1);
    printf("Candidate 2: %lu\r\n", vote2);
    printf("Candidate 3: %lu\r\n", vote3);

    ILI9341_FillScreen(WHITE);
    ILI9341_DrawText("BIOMETRIC VOTING", FONT6, 25, 10, BLACK, WHITE);
    ILI9341_DrawText("MACHINE", FONT6, 100, 30, BLACK, WHITE);
    ILI9341_DrawText("RESULT", FONT6, 110, 60, PINK, WHITE);

    char buffer3[50];
    char buffer4[50];
    char buffer5[50];
    sprintf(buffer3, "Candidate 1: %lu", vote1);
    sprintf(buffer4, "Candidate 2: %lu", vote2);
    sprintf(buffer5, "Candidate 3: %lu", vote3);

    ILI9341_DrawText(buffer3, FONT7, 80, 110, BLUE, WHITE);
    ILI9341_DrawText(buffer4, FONT7, 80, 140, BLUE, WHITE);
    ILI9341_DrawText(buffer5, FONT7, 80, 170, BLUE, WHITE);
}

void Clearing_Fingerprints(void)
{
    printf("\r\n=== Clearing All Fingerprints ===\r\n");
    if (R307_clearAllFingerprints() != HAL_OK){
        printf("Failed to clear All Fingerprints\r\n");
        ILI9341_FillScreen(WHITE);
        ILI9341_DrawText("BIOMETRIC VOTING ", FONT6, 25, 10, BLACK, WHITE);
        ILI9341_DrawText("MACHINE", FONT6, 100, 30, BLACK, WHITE);
        ILI9341_DrawText("Failed to clear All Fingerprints", FONT7, 10, 120, RED, WHITE);
        Error_Handler();
    }
    printf("All Fingerprints Removed\r\n");
    ILI9341_FillScreen(WHITE);
    ILI9341_DrawText("BIOMETRIC VOTING ", FONT6, 25, 10, BLACK, WHITE);
    ILI9341_DrawText("MACHINE", FONT6, 100, 30, BLACK, WHITE);
    ILI9341_DrawText("All Fingerprints Removed", FONT7, 30, 120, BLUE, WHITE);
}

void enroll_Fingerprints(void)
{
    for (int i = 1; i <= NUM_FINGERS; i++)
    {
        HAL_StatusTypeDef status;
        int numRetries = 3;
        do {
            char buffer6[50];
            sprintf(buffer6, "finger to enroll at ID %d", i);
            ILI9341_FillScreen(WHITE);
            ILI9341_DrawText("BIOMETRIC VOTING", FONT6, 25, 10, BLACK, WHITE);
            ILI9341_DrawText("MACHINE", FONT6, 100, 30, BLACK, WHITE);
            ILI9341_DrawText(buffer6, FONT7, 50, 120, BLUE, WHITE);
            printf("\n\nPlace your finger to enroll at ID %d", i);
            HAL_Delay(2000);
            status = R307_Enroll(i);
        }
    }
}

```

```

    } while ((status != HAL_OK) && (numRetries-- > 0));

    if (status != HAL_OK)
    {
        char buffer7[50];
        sprintf(buffer7, "for ID %d after 3 attempts", i);
        ILI9341_FillScreen(WHITE);
        ILI9341_DrawText("BIOMETRIC VOTING", FONT6, 25, 10, BLACK, WHITE);
        ILI9341_DrawText("MACHINE", FONT6, 100, 30, BLACK, WHITE);
        ILI9341_DrawText("Enrollment failed", FONT7, 50, 120, RED, WHITE);
        ILI9341_DrawText(buffer7, FONT7, 50, 150, RED, WHITE);
        printf("Enrollment failed for ID %d after 3 attempts", i);
    }
}

void Error_Handler(void)
{
    __disable_irq();
    while (1)
    {
    }
}

/* USER CODE END 4 */

```